

Projeto Integrador de Tecnologia da Informação III

Prof. Dr. Hudson Silva Borges

Módulo 4 – Integração e entrega contínua

Unidade 2 - Integração e entrega contínua
na prática

Práticas de DevOps

- Na unidade 1, vimos que **DevOps** é um conceito que visa unir Desenvolvimento (Dev) e Operação (Ops) com objetivo de agilizar a implantação de sistemas.
- **CI (Continuous Integration) e CD (Continuous Delivery)** são processos que permitem automação e monitoramento contínuo do ciclo de vida de aplicações.

- Em times com desenvolvedores com pouca experiência, a implantação destas práticas pode ser mais desafiadora, mas trazer muitos benefícios!
- Portanto, nesta unidade, o objetivo principal será a apresentação e a discussão de dicas e tecnologias que podem apoiar a implantação de CI/CD no projeto integrador desenvolvido.

Por onde começar?

- Comece com pequenas automações:
 - Identifique processos passíveis de automação.
 - Verifique a viabilidade de automatizá-las.
 - Busque ferramentas e alternativas.
 - Automatize a atividade.

Exemplo: conformidade de código

- Conformidade de código refere-se ao processo de definir e aplicar padrões predefinidos de qualidade, estilo, boas práticas, etc., no desenvolvimento de artefatos de *software*.
- Esses padrões permitem melhorar a manutenção e facilitam a colaboração, garantindo um código padronizado.
- Mas como garantir a conformidade?

Exemplo: conformidade de código

- Existem diversas ferramentas que apoiam desenvolvedores nesse processo, como, por exemplo:

Java



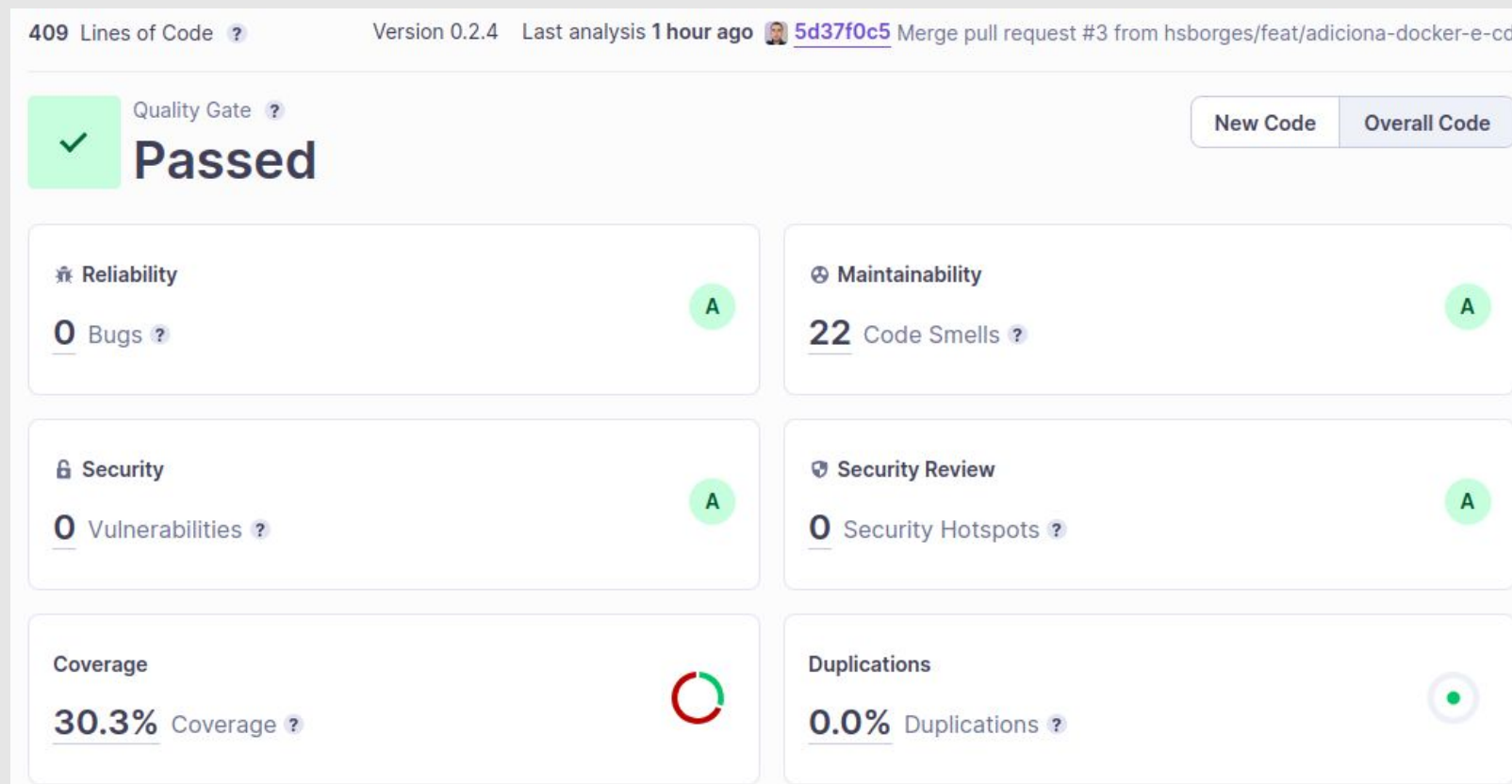
Python



JavaScript



Exemplo: conformidade de código



Fonte: Elaboração própria.
Relatório de verificação pelo sonarcloud.io

Exemplo: conformidade de código

- Podemos usar de ferramentas para automação de atividades de verificação, e até mesmo aplicação, de padrões de código e associá-las ao ciclo de desenvolvimento.
- Por exemplo, as seguintes ferramentas são locais:



[typicode/husky](https://typicode.com/husky)



Pre-Commit

[pre-commit/pre-commit](https://pre-commit.com/pre-commit)



gradle.org

Exemplo: conformidade de código

- Por fim, você também pode usar de serviços de integração contínua que executam as tarefas remotamente e notificam os resultados posteriormente, como, por exemplo:



Configuração do GitHub workflow

```
1  name: CI
2
3  on:
4    push:
5      branches: [main, pca ]
6    pull_request:
7      branches: [main]
8    workflow_dispatch:
```

Fonte: Elaboração própria.

```
10  jobs:
11    test:
12      runs-on: ubuntu-latest
13
14      steps:
15        - uses: actions/checkout@v3
16        - name: Use Node.js
17          uses: actions/setup-node@v3
18          with:
19            node-version: '20.x'
20        - name: Install dependencies
21          run: yarn
22        - name: Run test
23          run: yarn test
```

Resultado da execução do *workflow*



**Some checks were not successful**
1 failing and 1 successful checks

[Hide all checks](#)

	 CI / test (pull_request) Failing after 40s	Details
	 CI / test (push) Successful in 41s	Details
	This branch has no conflicts with the base branch Merging can be performed automatically.	

Merge pull request

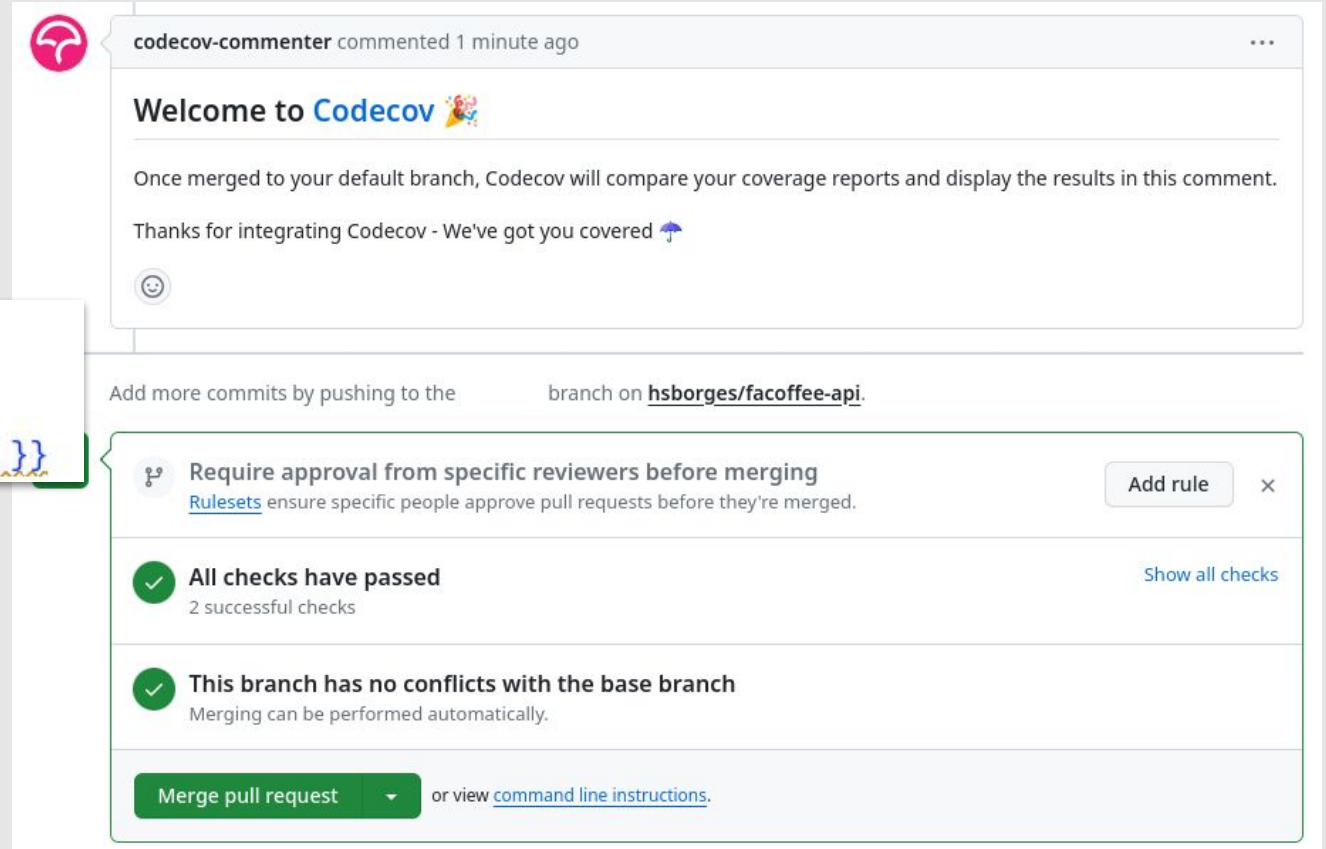
▼

or view [command line instructions](#).

Fonte: Elaboração própria.

Expandindo atividades do *workflow*

```
24 - name: Upload coverage reports to Codecov
25   uses: codecov/codecov-action@v3
26   env:
27     CODECOV_TOKEN: ${{ secrets.CODECOV_TOKEN }}
```



The screenshot shows a GitHub pull request interface. At the top, a comment from 'codecov-commenter' says 'Welcome to Codecov' and explains that coverage reports will be displayed in this comment. Below the comment, there's a section for 'Add more commits by pushing to the' branch on 'hsborges/facoffee-api'. The main part of the screenshot shows a list of checks: 'Require approval from specific reviewers before merging' (with a link to 'Rulesets'), 'All checks have passed' (2 successful checks), and 'This branch has no conflicts with the base branch' (Merging can be performed automatically). At the bottom, there's a green button 'Merge pull request' and a link 'or view command line instructions'.

Fonte: Elaboração própria.

Aplique automações incrementalmente

- Como dito anteriormente, busque identificar e automatizar processos de forma incremental a partir de atividades simples, como:
 - Conformidade de código;
 - Construção de versões;
 - Execução de testes;
 - E outras verificações.

Aplique automações incrementalmente

- Você se lembra dos testes projetados e construídos anteriormente?
- Você pode automatizar sua execução e obter *feedback* imediato sobre a qualidade de quaisquer mudanças em desenvolvimento!

Novas oportunidades com IA

- Novas ferramentas de IA para revisão de código integrada à fluxos de CI podem oferecer novas oportunidades de melhorias.



tasks-workflow/src/routes/find_task.py

Comment on lines +580 to +588

```
580 + def _get_task_model_from_guid_or_raise(task_guid: str, process_instance_id: int) -> TaskModel:
581 +     task_model: TaskModel | None = TaskModel.query.filter_by(guid=task_guid, process_instance_id=
582 +     if task_model is None:
583 +         raise ApiError(
584 +             error_code="task_not_found",
585 +             message=f"Cannot find a task with guid '{task_guid}' for process instance '{process_i
586 +             status_code=400,
587 +         )
588 +     return task_model
```

 **coderabbitai** bot 1 min ago Contributor ...

The function `_get_task_model_from_guid_or_raise` is a good addition for encapsulating the logic of retrieving a task model by GUID and raising an error if not found. This centralizes error handling and makes the code more DRY. However, ensure that the status code 400 is appropriate for all use cases where this function is called, as a 404 might be more suitable for not found errors.

```
-     status_code=400,
+     status_code=404,
```

► Committable suggestion

 **sambuilds** Now Contributor Author ...

Great catch! Just Fixed it.

Fonte: coderabbit.ai

Entregue rapidamente

- Em geral, **CD** segue princípios semelhantes e muitas vezes é suportado pelas mesmas ferramentas.
- Por exemplo, publicar uma imagem Docker do sistema.

```
8 jobs:
9   docker:
10     runs-on: ubuntu-latest
11     steps:
12       -
13         name: Set up QEMU
14         uses: docker/setup-qemu-action@v3
15       -
16         name: Set up Docker Buildx
17         uses: docker/setup-buildx-action@v3
18       -
19         name: Login to Docker Hub
20         uses: docker/login-action@v3
21         with:
22           username: ${ secrets.DOCKERHUB_USERNAME }
23           password: ${ secrets.DOCKERHUB_TOKEN }
24       -
25         name: Build and push
26         uses: docker/build-push-action@v5
27         with:
28           push: true
29           tags: ${ secrets.DOCKERHUB_USERNAME }/nome_da_imagem:latest
```

Fonte: Elaboração própria.

CI/CD como **prática integradora**

- Atividades de CI/CD são práticas voltadas à automação de processos para melhoria da qualidade do processo e do produto de *software*.
- Não faz sentido aplicar práticas automatizadas simplesmente porque outros usam. Busque cultivar uma cultura integrada entre desenvolvimento e entrega de qualidade.

Atividade de checkout

- Você deverá criar um plano de integração de CI/CD para o projeto em desenvolvimento na disciplina de Projeto Integrador.
- Descreva claramente os processos que serão automatizados e apresente os objetivos e benefícios esperados com essa ação.
- Reflita também sobre os desafios e impactos esperados na aplicação do processo.

Atividade de checkout: dicas

- Para além da automação das atividades, reflita sobre o impacto e como poderá contribuir para a melhoria da qualidade e agilidade no desenvolvimento.
- Busque usar ferramentas e plataformas conhecidas, com grande comunidade e com uma visão de longo prazo e escalável.

Referências

GITHUB, Inc. **Sobre a integração contínua com o GitHub Actions (GitHub Docs)**. Disponível em: <https://link.ufms.br/rffv> .Acesso em: 22 nov. 2024.

GITHUB, Inc. **Sobre a implantação contínua com o GitHub Actions (GitHub Docs)**. Disponível em: <https://link.ufms.br/Yhjvw>. Acesso em: 22 nov. 2024.

MUNIZ, Antonio et al. **Jornada DevOps**: unindo cultura ágil, Lean e tecnologia para entregar software com qualidade. 2. ed. Rio de Janeiro: Brasport, 2020. ISBN: 9786599062117.

VALENTE, Marco Tulio. **Engenharia de Software moderna**: princípios e práticas para desenvolvimento de software com produtividade. v.1, n.24, 2020. Disponível em: <https://link.ufms.br/bljNp> . Acesso em 20 dez 2024.

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

